DHA Suffa University

Final Year Project

Emotion Aware Emergency Dispatch System (EAEDS)

S31F25

Software Requirement Specifications

Submitted By

Abdullah Shaikh(ds221042)

Syed Murtaza Hassan(ds221026)

Tahir Mehdi(ds221011)

Syed Muhammad Baqar(ds221043)

Supervisor

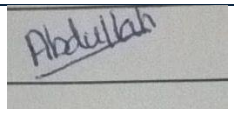
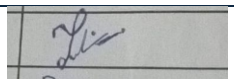
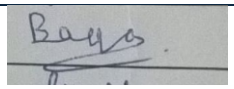
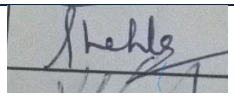
Ms. Shehla Andleeb

Co-Supervisor

Ms. Mashal Tariq

Document Sign off Sheet

Project Title	Emotion Aware Emergency Dispatch System (EAEDS)
Project Code	S31F25
Document Name	Software Requirements Specifications
Document Version	<1.0>
Document Identifier	S31F25-SRS
Document Status	Final
Author(s)	Abdullah Shaikh,Syed Murtaza Hassan,Tahir Mehdi,Syed Muhammad Baqar
Approver(s)	Miss Shehla Andleeb
Issue Date	27-10-2025

Name	**Role**	**Signature**	**Date**
Abdullah Shaikh	Team Lead		
Syed Murtaza Hassan	Team Member 2		
Tahir Mehdi	Team Member 3		
Syed Muhammad Baqar	Team Member 4		
Ms. Shehla Andleeb	Supervisor		
Ms. Mashal Tariq	Co-Supervisor		
	Project Coordinator		

Revision History

Date	Version	Description	Author
15/1/2026	1.0	Initial document	Abdullah Shaikh,Syed Murtaza Hassan,Tahir Mehdi,Syed Muhammad Baqar

Definition of Terms, Acronyms, and Abbreviations:

Term	Description
EAEDS	Emotion Aware Emergency Dispatch System, which is a system designed to analyze the emotions of callers and help dispatchers in their work.
LLM	A model that is train on alot of data like whole world data like chat gpt,gemini and many more.
Whisper	an open-source ASR model created by OpenAI that is used for converting speech into text.
BERT	Bidirectional Encoder Representations a model used for understanding natural language and analyzing emotions.
Open Smile	an open-source tool that extracts acoustic and emotional features from audio signals.
Mozilla TTS	an open-source TTS engine that generates speech that sounds like a real person.
Dispatcher	a person who handle 911 or emergency calls.
Caller	a person who calls for some sort of emergency.
Dashboard	the main page where live call information, emotions, and location of caller is displayed. In our project it is called midnight dashboard
Latency	the time it takes for a system to respond to an action.
API	used for communication between different softwares or backend to frontend.
GPU	Graphics Processing Unit, specialized hardware that speeds up AI calculations.
SQLite	Database for storing calls and other works
Next.js	React framework used for frontend in our project.
FastAPI	Python framework used in our project backend for creating apis.

Table of content:

Revision History 1

Introduction: 1

Purpose of Document: 1

Intended Audience: 1

Problem Statement: 1

Overall System Description: 1

Project Background: 1

Problem Statement: 1

Project Scope: 1
Not In Scope: 1
Project Objectives: 1
Stakeholders & Affected Groups: 1
Operating Environment: 1
System Constraints: 1
Assumptions & Dependencies: 1
External Interface Requirements: 1
Hardware Interfaces: 1
Software Interfaces: 1
Communications Interfaces: 1
System Functions / Functional Requirements: 1
System Functions: 1
System Functional Table: 1
System Attributes: 1
Use Cases: 1
Use Case Daigram: 1
References: 1
Use Case #1: 1
Use Case #2: 1
Non-Functional Requirements: 1
Performance Requirements: 1
Safety Requirements: 1
Security Requirements: 1
Reliability Requirements: 1
Usability Requirements: 1
Supportability Requirements: 1
User Documentation: 1

Introduction:

This document outlines the software requirements specification for Emotion Aware Emergency Dispatch System (EAEDS), an AI-powered agent designed specifically for the

(911 Emergency calls). The system aims to support dispatchers by taking the calls of people get stuck in emergency and talk with them like real human and get analysis of their emotions and show to live dashboard for dispatchers to take quick decisions for current emergency and this system also keep logs of recent calls for future problems.The software will be developed as a web-based platform and is intended to simplify the workflow of 911 dispatchers to take quick decision based on real-time information.

Purpose of Document:

This document includes all the software requirements for our project Emotion Aware Emergency Dispatch System (EAEDS). It includes the functional and non-functional requirements useful for developers, stakeholders, and quality assurance teams to understand the system's architecture ,uses and all core and non-core functions.

Intended Audience:

The purpose of this project is to develop an AI-powered agent that will takes calls and also talk like human and understand caller emotions and situation and response according to it and gather useful information that will used by dispatchers to take decision and lot of time will be saved because of this and lot of lives get saved during high natural emergency.

Problem Statement:

When some critical situation occurs in world like earthquake or any situation,lot of people call to 911 which creates mess in call centers due to which some callers get no answers due to less dispatchers therefore a lot of lives get wasted.

Overall System Description:

Project Background:

Emergency dispatch system are the first and the most essential line of defense in safety of public. However the current setups are mostly old, depending on physical work and human reasoning during hard times. The Emotion-Aware Emergency Dispatch System (EAEDS), also called dispatch Al, was made to enhance this procedure. By using nowadays advancements in Large Language Models (LLMs) and sound investigation the project aims to develop a smart dispatch helper. This helper can listen, understand and think along with human operators, changing raw audio into usable information.

Problem Statement:

Global emergency services are facing a lot of staffing problems, with reports showing 82% lack in 911 call centers. Human operators are burdened by high call sounds, which causes:

Increased Response Times: Postpones in managing information and sending units.

Human Error: High-stress environment tends to mismanagement or mental exhaustion.

Lack of context: Traditional systems depends on conversation with automatic devices to check call emotion straightforwardly. There is a crucial need for networking layers which can manage Early screening, record calls correctly and give decision support without changing human element.

Project Scope:

This project's range includes the design and creation of functional prototype for an Autonomous Dispatch Assistant. The main features of this include:

Real-time Speech-to-text: Instantly changes caller speech into text using Whisper model.

Emotion Detection: Inspecting vocal patterns with OpenSMILE to detect emotional states such as panic, fear, or neutrality.

Smart Decision-Making: Using fine-tuned LLMs to follow standard 911 processes (Medical, Fire, Police) and produce questions and suggestion according to current scenario of caller.

Operator Dashboard: It is a web based interface that is developed with Next.js and it represents live transcripts, detected emotions, and recommended actions for operators and call history.

Local GPU Architecture: A unified system where all AI processing and user interface components run on the local workstation equipped with NVIDIA GPU, providing faster response times and eliminating internet dependency for core operations.

Not In Scope:

The items which are clearly removed from this phase of the project are as follows:

PSTN/Telephony Integration: The system uses calls using WebSockets currently and is not connected to original landline or mobile networks currently.

Legal/Medical Liability: The system only provides recommendations and is not meant to make final legal or medical decisions without a human checking them.

Physical Hardware Deployment: This project is software based only and does not cover giving physical servers or specialized radio equipment.

Multi-Language Support: Our project only works with english language not with other languages so it is not multi language project.

Project Objectives:

Reduce Triage Time: Computerize the collection of details (Name, Location, Type of Emergency) to make dispatch faster.

Enhance Situational Awareness:Give operators live visual representers telling the caller's emotion and stress state.

Ensure Accessibility: AI assistance should be usable on standard local hardware equipped with a compatible NVIDIA GPU.

Operational Resilience: Make the system so that the interface stays responsive even though the AI experiences short postpones.

Stakeholders & Affected Groups:

Primary Users: 911 dispatchers and call takers who will work with the dashboard.

Secondary Users: First responders (Police, Fire, EMS) who receive the processed knowledge.

Beneficiaries: People or callers who gain rapid and more correct emergency response and fast service and risk of death will be less for people suffering from any critical emergency.

Administrators: Personnel responsible for managing the system, fine-tuning the AI model, and managing the servers.

Operating Environment:

he system operates on a single local workstation with the following requirements:

Hardware Requirements:

- Operating System: Windows 10/11 or Linux (Ubuntu 20.04+)
- GPU: NVIDIA GPU with minimum 6GB VRAM (GTX 1660 or higher recommended; RTX 3060/4060 or higher for optimal performance)
- CPU: Modern multi-core processor (Intel i5/AMD Ryzen 5 or higher)
- RAM: Minimum 16GB system RAM (32GB recommended)
- Storage: 50GB available SSD space for models and data
- CUDA Toolkit: Version 11.8 or higher

System Constraints:

Latency: Latency: Local GPU processing eliminates network delays but requires sufficient VRAM and processing power. Response time depends on GPU capability - minimum RTX 3060 recommended for sub-2-second latency

GPU Dependency: GPU Dependency: The system requires a dedicated NVIDIA GPU with CUDA support. Integrated graphics or non-NVIDIA GPUs are not supported. Minimum 6GB VRAM required for basic operation

Context Window: The language model has a range of memory for conversation so very long calls may lose early information until summarization is used.

Assumptions & Dependencies:

Dependencies: Dependencies: The system depends on external libraries (HuggingFace, PyTorch, Next.js, CUDA Toolkit) and services (Leaflet.js for maps). All AI processing dependencies are installed locally.

Assumption 1: Users will have a microphone available for simulating voice input.

Assumption 2: The local system has sufficient GPU resources (6GB+ VRAM) available for AI model inference.

Assumption 3: The internet connection that is provided will have enough speed of upload to transfer voice in real life time.

External Interface Requirements:

Hardware Interfaces:

Local Workstation: A GPU accelerated server (NVIDIA CUDA-compatible) is needed for low- latency inference of whisper and BERT models.

Client Workstation: Standard PC/Laptop with audio input/output capabilities for the dispatcher dashboard with good internet network.

Software Interfaces:

Backend: Fast API (python) serves as the important and core API and WebSocket handler.

Frontend Framework: Next.js (React) for the interactive Dispatcher UI.

Database: SQL Lite for efficient, lightweight storage of call logs, user profiles and transcripts.

Leaflet.js: For caller locations and display the tactical map view.

Open Smile and BERT: For Extracting semantic and acoustic features from audio calls.

Whisper: For converting our audio calls into text.

Mozilla TTS: used as a voice agent for communication with callers.

LLM or Ai Agent: Backbone of our project used as a brain to talk or give suggestions to our dialers or basically a replacement of dispatchers when they are busy.

Communications Interfaces:

This part explains how the different parts of the system share information with each other.

Communication Protocol (Client-Server): This is the main way for real-time data exchange between the Frontend and Backend.It handles live audio events, updated transcripts, and emotion data.

HTTP/REST (JSON): It is used for assignment that don't need to happen without the delay e.g. getting past call records (GET /calls), and checking if the system is performing it's function accurately(GET /), and saving the session information.

Data Exchange Format:JSON Objects: Includes all the data that is sent around like transcripts, emotion scores, and command triggers and it is arranged by using standard JSON format.

Internal/External API Calls: The Python backend communicate with the AI models by using async function calls if the models are present on the same machine.

Local Processing Pipeline: All AI components communicate via in-process Python function calls and shared memory, eliminating network overhead and ensuring sub-second response times

System Functions / Functional Requirements:

System Functions:

Speech Text Conversion: The system will convert incoming audio calls to text using whisper with latency of less than 2 seconds.

Dual Emotional Analysis: The system shall compute a weighted emotional score by combining Acoustic features with open smile and semantic features using BERT.

Automate Prioritization: The system will label calls as critical, high or low based on emotional intensity detected by our models better decision making by dispatchers.

Midnight Command Dashboard: The system will update the main page in real-time, displaying active calls and their transcript also and dynamic map markers without page reload.

Call Hand overing: The dispatcher can take over calls if condition is going critical.

System Functional Table:

The system functions for the Emotion-Aware Emergency Dispatch System are listed below, categorized by their priority and visibility to the end-user (Dispatcher).

Ref#	Functions	Category	Attribute	Details & Boundary Constraints
R1.1	Real-time Call Transcription(not on landline numbers)	Evident	Latency	Transcript appears < 2 seconds after speech via speech to text using whisper
R1.2	Emotion Classification Visualization	Evident	Accuracy / Refresh	Updates emotion gauge every 3-5 seconds based on audio accumulation
R1.3	Generate AI Response Suggestions	Evident	Relevance	Talk with caller via mozilla tts and generate responses according to its fine tuning
R1.4	Display Caller Geolocation	Evident	Precision	Display caller location for dispatchers via leaflet.
R1.5	Archive Call Logs & Transcripts	Hidden	Persistence	Automatically saves conversation history to SQLite upon disconnection for dispatcher records.
R1.6	Maintain WebSocket Connection	Hidden	Reliability	Handshake and heartbeat maintenance for bi-directional audio streaming
R1.7	Extract Acoustic Features	Hidden	Explainability	Runs OpenSMILE extraction on audio buffer chunks (background process) to analyze caller emotional feelings.
	Call Swicthing to dispatcher(If emergency is so critical)	Evident	Performance	If dispatcher wants to take call he can switch it.

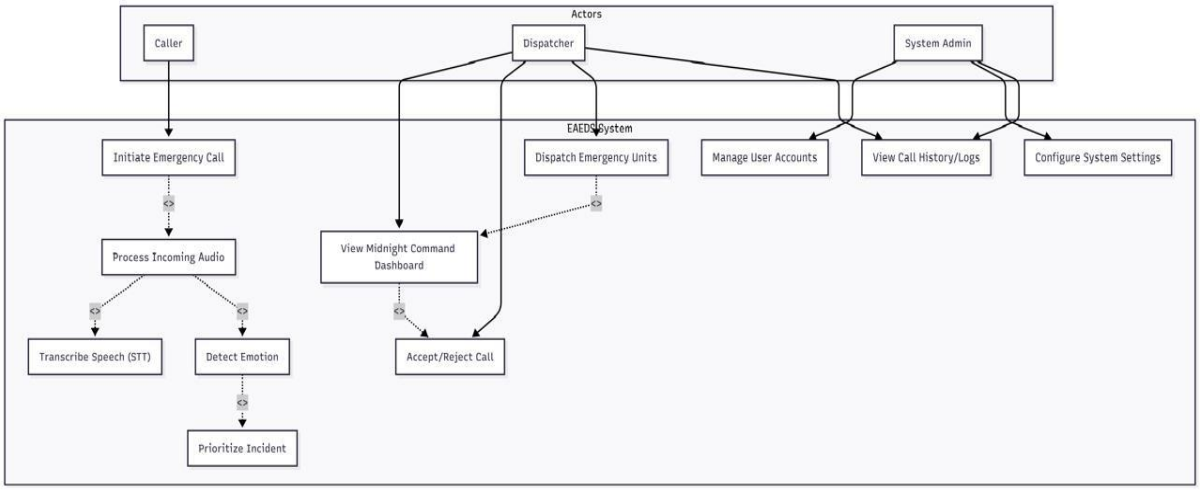
System Attributes:

Attribute	Details and Boundary Constraints	Category
Response Time	The time between when a caller finishes speaking and the text shows up on the dashboard needs to be short -under 2 seconds - so the dispatcher can read along in real-time.	Mandatory
Data Privacy	All audio processing and transcription must happen locally.No audio data should be sent to public third-party APIs (Privacy- first design).	Mandatory
Interface Metaphor	The dashboard must use a dark mode or high-contrast tactical interface that works well in low-light dispatch centers, helping reduce eye strain for 24/7 operations.	Optional(Depends on person to person)
Concurrent User Load	The system must handle at least 5 simultaneous mock calls during without the backend crashing.	Optional(For Demo in front of jury)

Use Cases:

Use Case	Title	Description
UC-01:	Monitor Live Call	The dispatcher can see the live text of the conversation, the emotional state of the caller, and their location during an ongoing emergency call.
UC-02:	Review Call History	The dispatcher can check the database to read previous call records and listen to the details of past emergency calls again.

Use Case Daigram:



References:

Reddy, S., & Kompalli, S. (2022). Emotion Detection using Multimodal Analysis. Journal of AI & Ethics.

Rajput, D., & Singh, H. (2023). BERT for Emotion Classification. Procedia Computer Science.

Huang, J., et al. (2020). Whisper and ASR Systems. arXiv preprint.

Open SMILE documentation. AudEERING GmbH (2020).

Use Case #1:

Section: Main	
Name:	Monitor Live Call (UC-01)
Actors:	Dispatcher, AI Agent
Purpose:	To give the dispatcher better awareness during an emergency and also caller dont have to wait if all dispatcers are busy.
Description:	When a call comes, system will detect it and talk with user and display all live transcript on main dashbaord and also label user emotions for dispacther and also show live location on map.
Cross References:	Functions: R1.1, R1.2 and R1.4
Pre-Conditions	The Backend and all ai models need to be active and connected also.
Successful Post-Conditions	The call ends, and all the conversation details are stored in the database.
Failure Post-Conditions	The transcription stops, and the system shows an error message that says "Connection Lost"

Typical Course of Events			
Actor Action		**System Response**	
1	The use case starts when the caller connects and the caller speaks (e.g., Help, there's a fire)	2	the AI Agent listens to the audio, converts it to text, and sends it out. and the AI Agent analyzes the sound and changes the "Emotion Gauge" to "High Urgency".
3	The dispatcher reads the text and checks the map.	4	The system keeps updating the screen as more audio comes in.. and Ai agent replying the user via mozilla voice agent also
5	The dispatcher takes action until the call ends.	6	The system saves the full conversation to SQLite (Function R1.5).

Alternative Course	
Step 4:	If the transcription is delayed, the system will show a message for dispatcher so that he knows there is some sort of delay.
Step 5:	If no voice is heard for more than 10 seconds, the AI Agent changes the emotion status to "Neutral/Waiting"

Use Case #2:

Section: Main	
Name:	Review Call History (UC-02)
Actors:	Dispatcher
Purpose:	To let the dispatcher look back at previous calls for reports or checks.
Description:	The dispatcher goes to the History tab, picks a call from the list, and sees the whole conversation, along with the final emotion result and where the call was made.
Cross References:	Function: R1.5
Pre-Conditions	There needs to be at least one call that has been finished and saved in the database.
Successful Post-Conditions	The details of the selected call appear on the screen.
Failure Post-Conditions	If the database isn't connected, the dispatcher will get no call history

Typical Course of Events			
Actor Action		**System Response**	
1	Dispatcher clicks the "Call History" button.	2	System checks the SQLite database for the last 50 records and then shows a list of calls with dates and phone numbers.
3	Dispatcher chooses a specific call.	4	System gets the full JSON conversation for that call and then show with labels like caller chat and agent chat
5	Dispatcher reads the chat and exits the view.		

Alternative Course	
Step 2(Empty Databse):	If there are no calls, the system shows a message that says "No prior calls found".

Non-Functional Requirements:

Performance Requirements:

Speech Transcription Latency: The system must transcribe audio as soon as it is received, and it needs to do this within 2 seconds to keep up with the event as it happens.

Emotion Inference Speed: BERT and Open Smile will identify emotions from the audio and send the results to the dashboard within 3 seconds of receiving the audio.This allows dispatchers to assess the caller's distress level in real time.

Safety Requirements:

Fail Safe Mechanism: The dispatcher will handle the call without any service interruption, if the system breaks down or there is any delay or problem in the AI pipeline. The system will be changed to Manual Dispatch Mode, when the AI services are not available.

Right Location Verification: Every reverse geocoded address must be tested against a trusted mapping service like the Map Box. This will make sure that first responders are scheduled to the real and original location with an accurate measurement of or within 50 meters of the recorded location.

Security Requirements:

Encryption At Rest: The database must be protected using strong encryption, such as AES-256, to keep caller information private.Important call recordings and transcripts should be stored in an encrypted database.

Authentication: "Midnight Command" dashboard is protected by a JSON Web Token (JWT).Session tokens will be expired after 12hours of usage..

VoIP Security: To prevent unauthorized access to emergency conversations, SIP signaling and RTP media streams will be secured using TLS and SRTP, respectively.

Reliability Requirements:

System Availability: The main models and telephone parts need to be available 99% of the time to ensure the line is ready for emergency calls.

Automatic Recovery: A process control system (Docker) should manage the Fats API backend (Docker) which can restart its working procedures in 5 seconds if they are not working or they stop operating .

Usability Requirements:

UI Design: The “Midnight Command” dashboard must have a dark mode color scheme that is present with clear alert presenters which will help to reduce eye strain during long and late night shifts.

Single Critical Page: The interface should must represent the live location on a map, along with a live transcript and emotion graphs. They all should be on one screen so operators can easily have access to significant situation details and can decide accurately and can make better decisions.

Supportability Requirements:

Modular Architecture: A loosely connected Microservice architecture (FastAPI) should be used to built the system so that individual AI models, like switching from BERT to Roberta, can be improved or changed without the need of reconstruction of the whole system or can cause any issue or downtime.

Deployment Consistency: Our project have to deploy and use docker containers to make sure that development,testing and production environment are same and has no overlapping or conflicts.

User Documentation:

The following documentation must be included with each system release:

Dispatcher User Manual: A guide that explains all the features used in main midnight dashboard and explain overall features of our project.

System Administration Guide: Technical instructions for handling database backups and deploying the Docker containers.